

Project - FileSystem

1. 概述

本 Project 的目标是在 [SUSTechOS](#) 上实现一个Ext2 文件系统，并将其接入到VFS（虚拟文件系统）和 virtio-blk驱动中，向上为用户进程提供统一的接口。

评分组成：

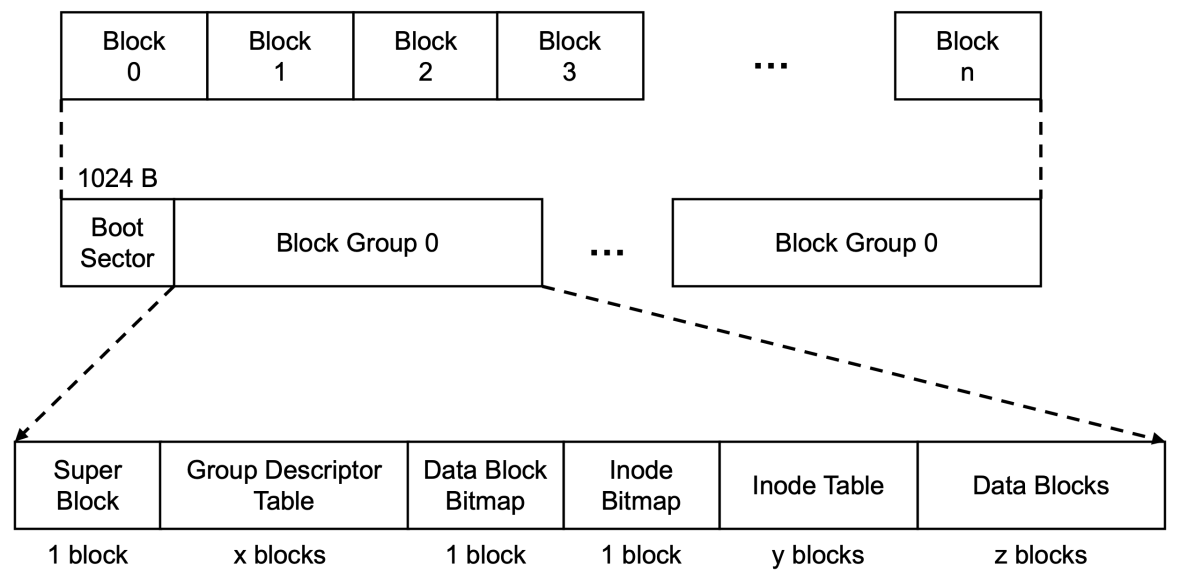
- 15分基础分，实现Ext2文件系统的功能，分数由checkpoint验证 + 答辩问答组成
- 5分bonus分，包括日志系统和性能优化，分数在答辩问答时确定

2. Ext2介绍

2.1 Ext2简介与结构

Ext2（第二代扩展文件系统）是Linux内核于1993年引入的一款经典磁盘文件系统。它采用“块组”结构来组织磁盘空间，即将整个分区划分为多个连续的块组，每个块组内部都包含自己独立的超级块、Inode表、位图和数据块区域。作为Ext系列演进的基石，Ext2为其后续版本奠定了核心架构：Ext3在其基础上主要增加了日志功能以提升崩溃恢复能力；而现代广泛使用的Ext4则进一步扩展，支持更大的存储容量、更优的性能特性（如区段扩展、多块分配）以及更丰富的功能。

Ext2文件系统以“块”作为数据存储与分配的最小单元，块大小可在创建文件系统时配置，通常为1KB、2KB或4KB。一个完整的Ext2文件系统在磁盘上的物理布局如下图所示：

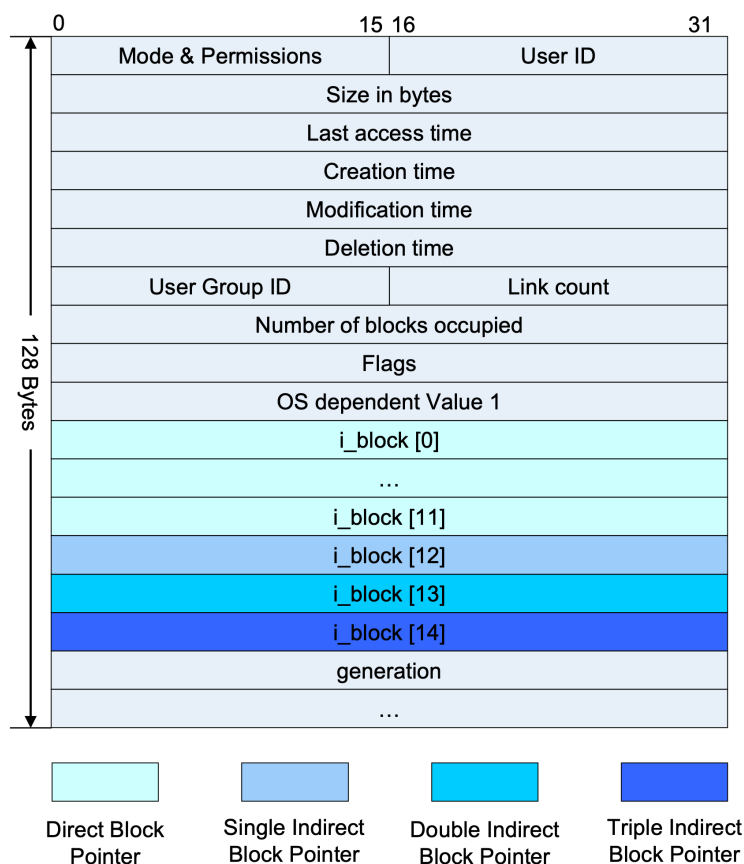


如图所示，Ext2文件系统起始部分是一个启动块，占用1024字节，随后是连续排列的多个块组。每个块组包含以下部分：

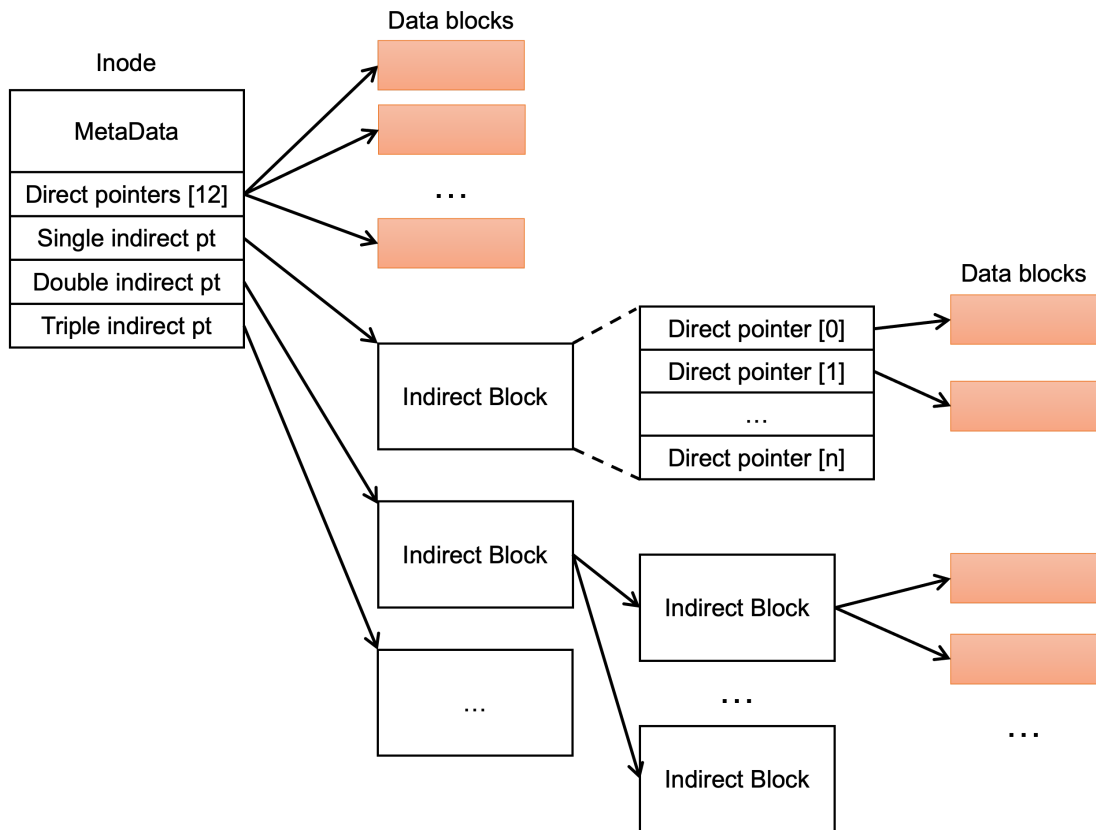
- **超级块**：存储整个文件系统的全局元数据，如块大小、块组总数、空闲块/Inode计数等。第一个块组（块组0）的超级块位于固定偏移1024字节处，其余块组中的超级块是其备份，用于灾难恢复。
- **块组描述符表**：一个由多个块组描述符组成的数组，每个描述符记录了对应块组的关键信息，例如块位图和Inode位图的起始块号、Inode表的起始块号等。
- **块位图**：一个比特位图，用于跟踪本块组内所有数据块的空闲与已使用状态（1个比特代表1个数据块）。
- **Inode位图**：一个比特位图，用于管理本块组内所有Inode的空闲状态。

- **Inode表**：一个连续存储的Inode数组，每个Inode的大小固定为128字节，用于存储文件或目录的元数据。
- **数据块**：实际存储文件或目录项列表的磁盘空间。

其中的核心概念**Inode**（索引节点）是Ext2管理文件实体的关键数据结构。每个Inode固定为128字节，包含了文件/目录的所有元信息，例如类型、权限、所有者、大小、时间戳（创建、修改、访问时间）以及指向其数据块的指针数组。其结构示意图如下（翻译自：[知乎](#)）：

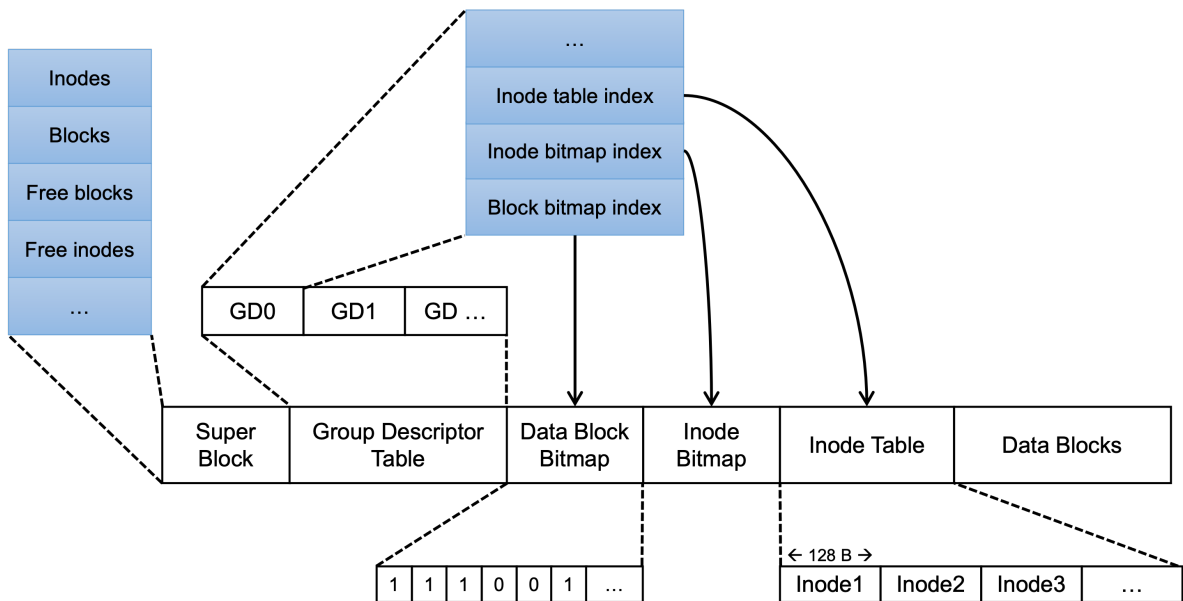


图中最后的 `i_block[xx]` 数组即数据块指针。**前12个为直接指针**，直接指向存储文件数据的数据块；**第13个为一级间接指针**，指向一个包含更多直接指针的块；**第14个为二级间接指针**；**第15个为三级间接指针**。其多级索引原理如下图所示：



2.2 Ext2初始化流程

操作系统挂载一个Ext2文件系统时，会执行一系列初始化操作以建立其内存中的管理结构，Ext2中块组初始化的相关结构如下：

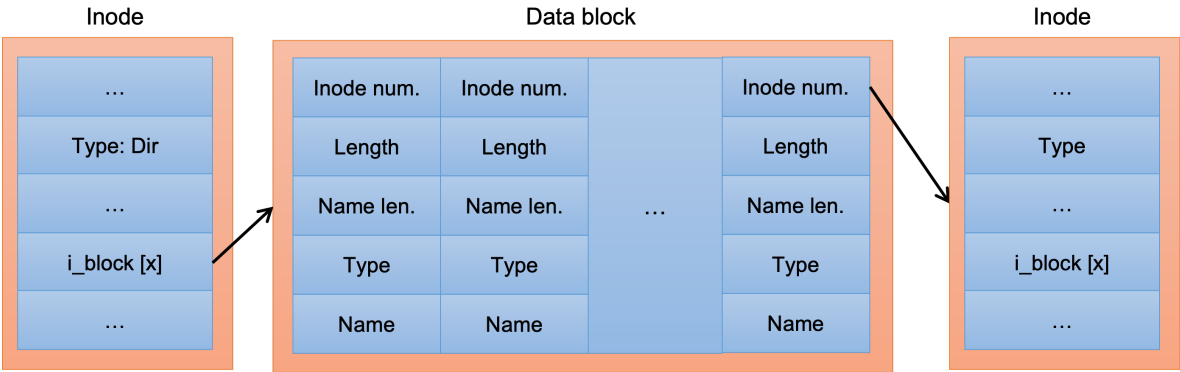


对应的过程如下：

1. **读取超级块**：操作系统首先从磁盘的固定偏移（第一个块组的1024字节处）读取**超级块**。通过验证其中的魔法值（0xEF53）来确认这是一个有效的Ext2文件系统。
2. **加载块组描述符表**：根据超级块中的信息，定位并读取整个**块组描述符表**到内存。该表提供了所有块组关键元数据，如位图和Inode表位置。
3. **建立缓存**：将超级块与块组描述符缓存进内存中，并根据块组描述符，索引到块位图、Inode位图与Inode表。

2.3 Directory的存储与索引

在Ext2中，**目录（Directory）**是一种特殊类型的文件，其Inode类型标记为 `S_IFDIR`。目录的结构与索引如图所示：



当用户希望查看一个文件（如 `/home/user/file.txt`）是否存在时，操作系统通过以下步骤索引到该Inode：

1. **路径解析**：从根目录（默认为Inode 2）开始。读取根目录的Inode，根据其 `i_block[]` 指针找到存储根目录项的数据块。
2. **目录项遍历**：在根目录的数据块中，线性扫描目录项，查找名为“home”的条目，并获得其Inode号。
3. **递归查找**：读取“home”目录的Inode，进而读取其数据块，查找名为“user”的目录项，获得其Inode号。重复此过程，最终在“user”目录的数据块中找到名为“file.txt”的目录项，并获取目标文件的Inode号。

2.4 Ext2的读取与写入操作

Ext2 的文件读取操作较为简单：根据目录索引到目标文件后，利用 `i_block[xx]` 即可读取文件数据。

Ext2 的文件写入则稍复杂：若**不需要扩展文件大小**，只需覆盖目标数据块的内容，并更新单个 Inode 的元数据；若**需要扩展文件大小**，则需额外执行从块位图中查找并分配空闲数据块，以及更新全局元数据（如超级块等）的操作。

2.5 Ext2到Ext4的演进

Ext2 文件系统作为 Linux 早期广泛使用的格式，虽然稳定可靠，但在性能、可靠性和功能上逐渐显现出局限性。为适应不断增长的需求，Ext2 经历了两次重要升级：Ext3 与 Ext4。

Ext2 --> Ext3

Ext3 在 Ext2 基础上最核心的改进是引入了**日志（Journaling）功能**。在 Ext2 中，若系统在写入文件元数据（如 Inode、目录项）或实际数据时突然断电或崩溃，文件系统可能因部分更新未完成而处于不一致状态。此时必须通过耗时的全盘检查（`fsck`）来扫描并修复错误。

Ext3 的日志机制解决了这一问题。其原理是：在实际执行磁盘写入操作之前，先将这些操作的“意图”记录到一个专用的日志区域。这份日志如同一份操作清单，若所有操作顺利完成，相应的日志条目会被标记为完成并清理。若发生崩溃，则在下一次挂载时，系统只需读取并分析日志，然后根据记录重放（Redo）或撤销（Undo）相关操作，即可快速、安全地将文件系统恢复到崩溃前的一致状态，从而避免了全盘扫描的巨大开销。

Ext3 提供三种日志模式，以在安全性与性能之间进行权衡：

- **日志模式**：记录所有元数据与文件数据，安全性最高，但性能开销最大。
- **顺序模式**：仅记录元数据，并确保数据块先于元数据写入，平衡安全与效率。
- **回写模式**：仅记录元数据，且不严格保证写入顺序，性能最优。

Ext3 --> Ext4

Ext4 是对 Ext3 的一次全面升级，旨在突破其在超大容量与高并发场景下的性能瓶颈。

Ext4 的一项核心改进是引入**区段 (Extents)**，用以取代 Ext2/3 中传统的块映射方式。在 Ext2/3 中，大文件可能需要通过大量间接指针块来映射成千上万个数据块，导致元数据庞大且容易产生碎片。区段则将文件中连续的数据块描述为一个范围（例如“起始块号 1000，连续 200 个块”，其概念类似于内存管理中的“起始地址+长度”），仅需一个区段结构即可表示，从而大幅减少了元数据量。

此外，Ext4 还实现了多项分配策略：

- **多块分配**：允许一次性分配多个连续的数据块，减少频繁分配操作的开销与碎片产生。
- **延迟分配**：将数据块的实际分配决策推迟到数据即将写回磁盘的时刻，从而积累更多的写入请求，实现更优的连续性分配。

3. 评分要求

本次评分点中，通过checkpoint只占40%，答辩问答占60%

3.1 基础

[4 pts] 挂载与超级块解析：系统需从磁盘固定偏移（第一个块组的 1024 字节处）正确读取超级块（Superblock）。根据超级块信息，正确加载块组描述符表（Block Group Descriptor）到内存，并建立位图和 Inode 表的缓存。

[5 pts] 文件创建与读写：支持基本的文件操作指令：`read`, `write`, `touch`，并且针对小文件写入，需能正确覆盖目标数据块内容。

3.2 目录与索引

[2 pts] 目录创建、查找与删除：

- 支持 `mkdir`, `rmdir`, `get_dents` 等目录操作。
- 需正确识别类型标记为 `S_IFDIR` 的目录文件，并支持从根目录（Inode 2）开始的路径解析。
- 能够读取目录数据块，定位目标文件的 Inode 号。

[2 pts] 直接与间接索引：

- 实现对 `i_block[xx]` 指针数组的解析，必须支持前 12 个直接指针的数据读写。
- 支持大文件读写，正确处理第 13 个（一级）、第 14 个（二级）和第 15 个（三级）间接指针的块映射原理。

3.3 元数据与系统规范

[1 pts] 元数据的正确更新：

- 在文件写入或扩展文件大小时，必须正确且及时地更新 Inode 中的关键元数据（如文件大小 `file size`、访问时间 `access time`、修改时间 `modification time` 等）。
- 涉及文件扩展时，需正确执行从块位图中分配空闲数据块的操作，并同步更新超级块等全局元数据。

[1 pts] 接口规范：

- 需要接入系统的VFS，实现的 Ext2 除了能读写普通文件，还必须能在系统中包含 FIFO（Named PIPE）类型的特殊文件。

4. Bonus (共5分附加分)

本部分内容旨在鼓励同学们探索现代文件系统的高级特性，并优化文件系统性能。

4.1 日志系统到 Ext4 (1 pts)

将基础的 Ext2 文件系统向 Ext3 或 Ext4 演进，实现基础的日志系统：引入日志（Journaling）区域，记录磁盘操作意图。

注：基于日志系统往Ext4发展也能对后续的性能测试有一定优化，比如引入区段（Extents）结构取代庞大的间接指针块映射，减少元数据碎片；或实现多块连续分配与延迟分配策略，降低频繁分配造成的开销。

4.2 性能测试 (4 pts)

本项 Bonus 将在最终答辩时执行性能测试，并根据排名情况给出具体的分数（最高 4 分），包含以下测试点：

- **大文件吞吐量**：测试数据的顺序读取与顺序写入性能。
- **随机 I/O 性能**：测试数据的随机读取和随机写入性能。
- **元数据与小文件操作**：测试小文件的创建、写入、读取和删除的延迟表现。

5. 答辩与提交要求

最终答辩需要展示在 Project 中的实现思路、实现细节，我们会对你代码中的某些部分进行提问，预计安排在第15周（6月1日~6月5日）。**答辩不以ppt的形式进行，而是通过现场问答，通过checkpoint只占40%的分数，答辩问答占60%，请务必理解代码逻辑。**

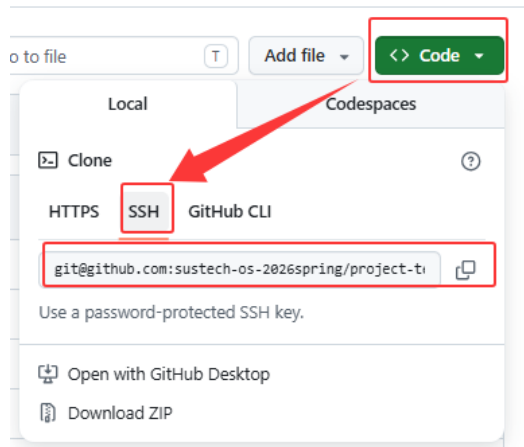
本学期将会使用github classroom作为代码仓库（邀请连接：<https://classroom.github.com/a/JGNFCPZR>），没有注册GitHub的同学请注册一个，进入教室并绑定自己的学号，如果学号已绑定请联系助教，此外，请务必绑定自己学号的邮箱到Github并作为公开邮箱，我们将会在答辩周问答时检查一遍学号和GitHub账号的绑定情况：



除代码仓库外，请以小组的形式在blackboard上传PDF形式的报告，报告需要包含小组成员列表，project设计架构，实现细节以及通过测试的截图。

我们在仓库中提供了基础代码，请各位同学在基础代码上完成project。使用流程如下：

1. 通过邀请链接加入班级，请在邀请链接中选择自己对应的学号（邀请连接：<https://classroom.github.com/a/JGNFCPZR>）
2. 队伍中1人创建队伍，其余队员加入队伍。每支队伍需要有三名成员，原则上三名成员需同属于同一实验班，如有成员属于其他实验班，则需要提供教务系统的课表证明在其加入的队伍所在的实验班时段无其他课程。
3. 加入队伍后，会自动生成队伍仓库。
4. 在实验环境（ubuntu）中参照 GitHub_SSH_Key.pdf 文档配置密钥。
5. 通过仓库右上角code->ssh复制git clone的链接，在实验环境中将仓库代码git clone至本地



6. 阅读仓库中的README文件，并完成project。

7. 完成后的代码提交至仓库，报告pdf提交至bb站点，bb提交窗口会在组队完成后开放。

AI使用说明

本课程**鼓励**在完成 Project 期间使用各类生成式 AI 工具（如 ChatGPT、Gemini、DeepSeek 等）辅助学习与开发，但请遵守以下规范：

- **代码生成声明**：如果在代码中使用了AI工具生成了代码，必须在提交时进行声明。
- **提交附带文档**：请在项目根目录下新建一份名为 `AI_USAGE.md`（或 `.pdf`）的文档，并附上生成该段代码时对应的上下文对话记录（包含 Prompts 以及 AI 的 Responses）。
- **代码理解**：大型语言模型在生成操作系统代码时，极易产生“幻觉”或逻辑漏洞。**请务必审查 AI 生成的代码，确保理解代码的运作原理，此外，若在最终答辩进行代码 Review 时无法解释自己提交的代码，将面临扣分风险，最终分数将由问答和测试共同决定，其中问答为重要考虑因素。**

注：

1. 说明使用AI工具并不会对最终成绩产生影响，不会因为手写代码和AI生成代码而产生评分区别，但要注意代码理解。
2. Project的AI要求和作业要求不一样。